ENGG-1410 Mini Project 1

TicTacToe

2026.11.02

Group 142

Luke Dykstra (1440689) & Deven Mathoo (1423575)

Section 08

Section 1: Problem Statement

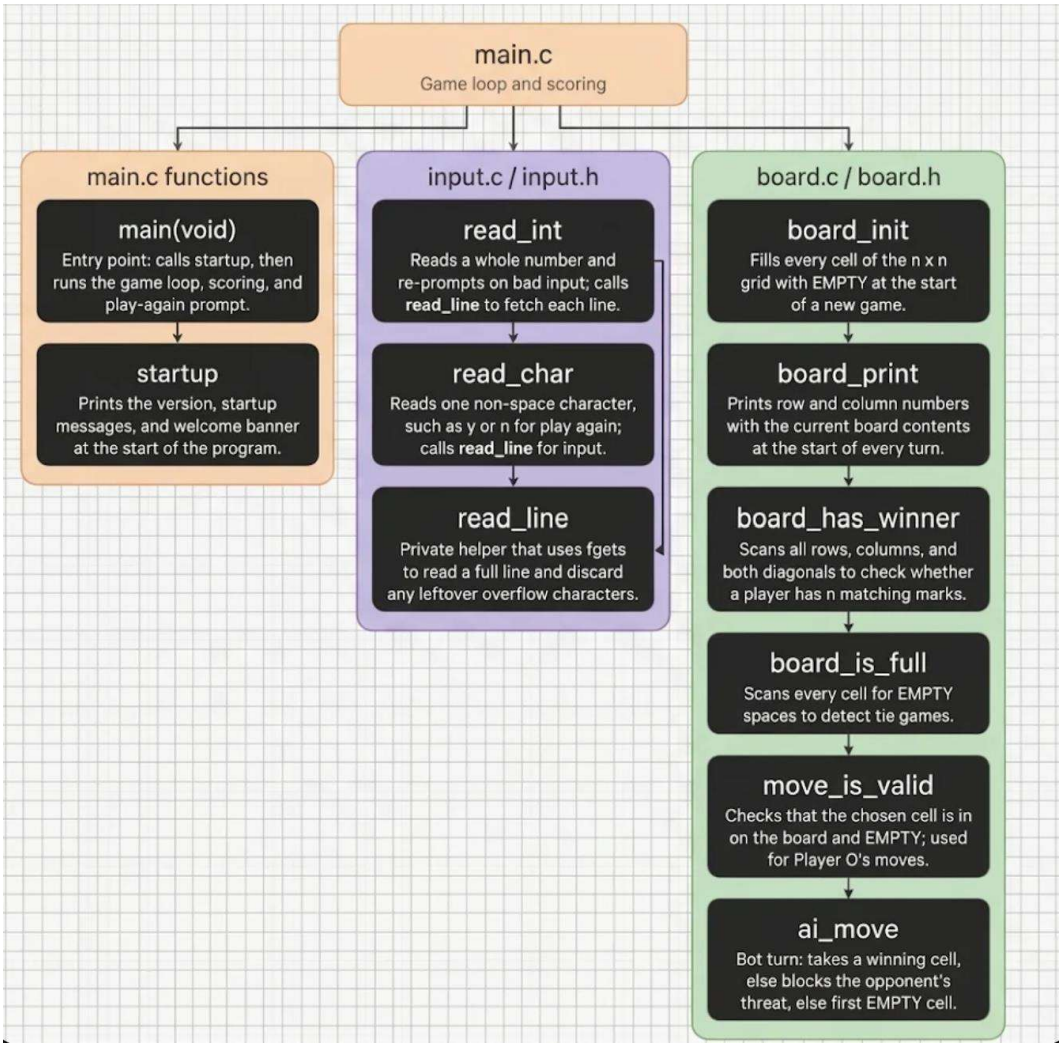
Problem statement: what the program does, in your own words.

The program is a scalable Tic-Tac-Toe game programmed in C. It allows users to configure a custom grid size of N ranging from 3x3 to 10x10, where the win requirement is to have 1 player (computer or user) match N number of their symbols in a row; horizontal, vertical, or diagonal. There are 2 modes of play: a 2-player mode where users alternate turns until a win condition is met, and a 1 player mode against a computer opponent. The computer opponent plays optimally with the goal of winning, until it detects a winning move by a user and will block the opponent's winning moves. The program also uses continuous game loops; tracks running scores for wins and draws across many rounds and will quit upon request after a game is completed.

Section 2: Design

2. Design: a block diagram showing which function calls which, and pseudocode for ai_move. Justify one design choice you made where there was a reasonable alternative.

Diagram detailing which functions call on each other:



Pseudocode for ai_move.

This function has the computer scan for if it is one moves away from winning, by scanning all possible winning line rows, columns, diagonals, and antidiagonals. If it finds such a single empty cell, it then places its symbol mark there to make a solid line of its marks and score a win. Then it exits.

If no winning move is found, the code moves to the bot, checking all lines again for if the player is one move away from winning. If the player is one move from winning, then it will place its mark in the empty cell to block them and then exit.

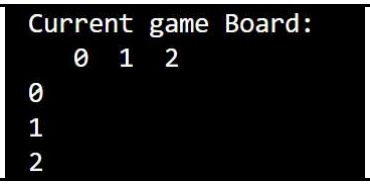
If neither the computer or player is one move from winning, then the computer will default to the simple strategy of scanning the boards' rows from top to bottom and place its mark in the first empty cell, then exit.

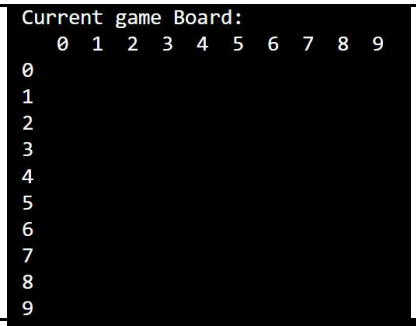
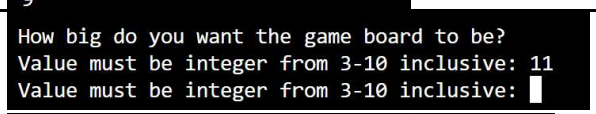
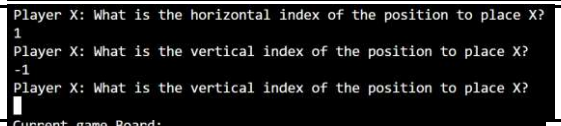
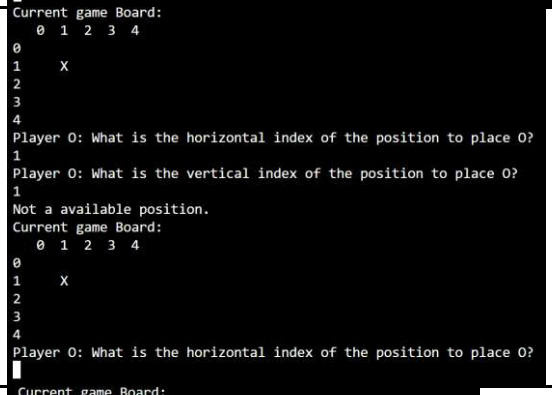
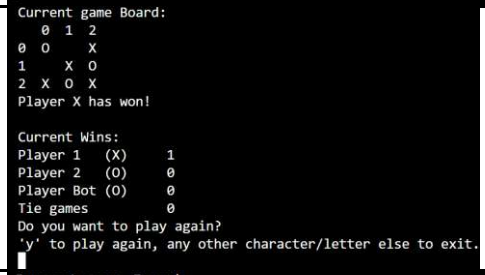
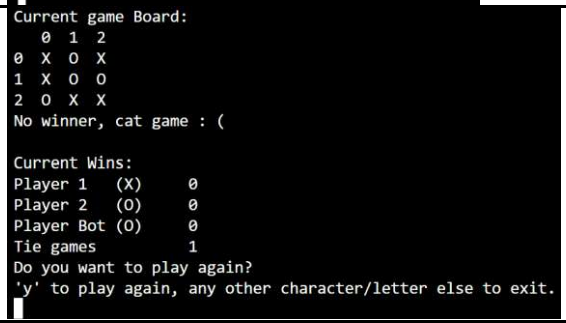
Design Choice alternative:

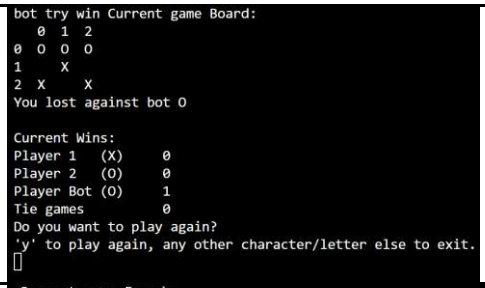
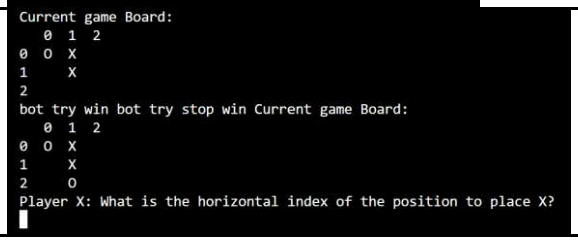
The computer detects wins by counting the marks in each row, column, diagonal, and antidiagonal, then comparing them against N. An alternative was a method that involved rotating the board 90 degrees clockwise around the center 4 times to evaluate each position. This worked for the N size of 3 but could not work with any of the other larger board sizes from N being 3 to 10. Therefore, this method was abandoned, and a direct counting approach that adapted to any N size that still checked for N-1 marks was used.

Section 3: Testing

3. Testing: a table of the test cases you ran, each with the input, the expected result, and the actual result. It must include at least these nine: size 3, size 10, an invalid size, an off-board move, an occupied cell, a diagonal win, a draw, the computer taking a win, and the computer blocking.

Test #	Test Case	Input	Expected Result	Actual Result
1	Size 3	Size: 3	Board set to 3x3 grid	

2	Size 10	Size: 10	Board set to 10x10 grid	
3	Invalid size	Size: 11	Reject input, and re-prompt users.	
4	Off-Board move	Row: 1, col: 1	Reject move as occupied and re-prompts user	
5	Occupied Cell	Play on row 1, col 1 when it is already marked.	Reject move as occupied and re-prompt user	
6	Diagonal Win	Align a symbol along a diagonal	Detect win, announce winner, increase their score by 1, and prompt the next round.	
7	Draw	Fill all cells with no symbol in a row to force a draw.	Detect full board, announce draw, increase draw count. Prompt user option to play again	

8	Computer Win	Set up board where the computer can align its symbols to win	The computer places its last symbol on board and detects its victory. Increase player bot by 1 and prompt the next game.	
9	Computer Block	Set up board to where player is about to win against computer	The computer detects the win threat and places its symbol to block the player.	

Section 3: Error Analysis

4. Error analysis: at least two bugs you encountered. For each: the symptoms you saw, how you located it (compiler warning, gdb, print statements, reasoning), and the fix. "It didn't work so I rewrote it" is not an error analysis.

1. When attempting to compile the project, running the makefile immediately encountered an error. The output was: “make: *** No rule to make target 'board.c', needed by 'board.o'. Stop.” This was located via logical reasoning as the text indicated that the makefile could not find the file board.c. Knowing this the board.c file was moved into the correct project folder with the makefile and the other files, so that makefile would build correctly, and then run without errors.

2. During testing we found that when the computer blocked it would respond with “bot try win bot try stop win” when it was only supposed to print bot try stop win. This was inside the ai_move function, where the print command for “bot try win” and “bot try stop win” were at the top of their respective check block, and bled into each other. This meant when the computer win condition scan was done it printed bot try win automatically even if it didn't see a win condition, so that when the next scan of player win condition was met it printed both “bot try win” and “bot try stop win.” To fix this, the print statements were moved inside the conditional success checks, so that they only printed after a valid winning or blocking move was found and verified. This “bot try win” and other text was only for debugging purposes to help visualize what was happening during the program running. Because this was only for debugging, that text was removed in the final versions.